Generalized columns in Htop

Personal Information:
Name: Sohaib Mohamed
University: Misr University for Science & Technology
Email: sohaib.amhmd@gmail.com
GitHub: https://github.com/smalinux
Location: Giza, Egypt

Why do I think I could implement this task

Because I have experience with PCP/Htop and I implemented similar parts:
DynamicMeters: https://github.com/htop-dev/htop/pull/669
DynamicColumns: https://github.com/htop-dev/htop/pull/707

DynamicTabs Description:

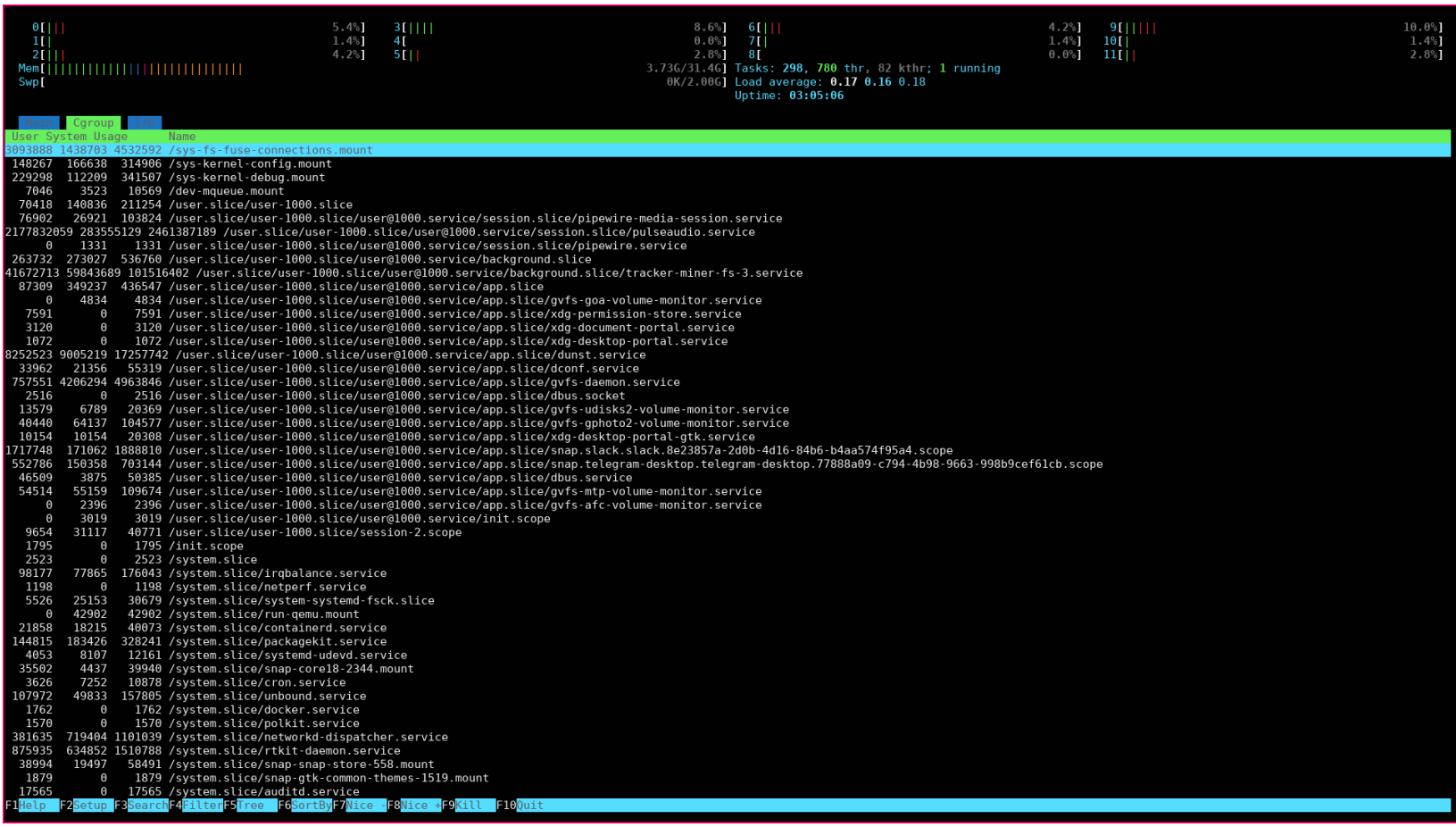
The Htop utility recently acquired the ability to display multiple Tabs in its user interface. Currently, these display only process information. The aim of this project is to support generalized Tabs, where information beyond processes can be displayed in columns.

it'd need to specify certain things including (importantly) a 'key' such that when the tab is displaying metrics that are not proc.* we know what we're working with (in other words, we want to remove the requirement that Htop columns can only show processes with a primary key on PID)
Classic example of this would be being able to display cgroups on Linux, see 'pminfo cgroup'
It's needed to get to a place where we could start creating tabs that can show resource utilization of things like this (i.e. cgroups, and other PCP instance domains other than proc)

Fetch any PCP metrics

Fetching any metric & instance value is a relatively easy task because of PCPMetrics helper functions.
I already wrote some drafts to fetch `cgroup.cpu.stat.user`, `cgroup.cpu.stat.system, cgroup.cpu.stat.usage` metrics and I

could print their values into the Htop UI.
See git branch: https://github.com/smalinux/htop/commits/DynamicTabs_101
Try to compile it and insert the ‘cgroup’ column, This column on the most left is coming from `cgroup.cpu.stat.user`:



Generic struct instead of Process

I think this is the most tricky part of the whole project.
Htop has a big struct called `typedef struct Process_` This holds each piece of process information like PID, PPID, TGID,... etc
This struct is considered to be the cell that makes up the Htop body.

Despite all of that. It's not suitable for our case. There are a lot of PCP metrics that don't base on the PID concept. like cgroup, bpf, filesys, etc...
So I need to introduce another big vector or hashtable to these PID-less and generic PCP metrics.

Once PCP/Htop starts, It's allocated `struct ProcessList_`
One instance of ProcessList represents all Rows and columns of the PCP/Htop.

```
PCPProcessList* this = xCalloc(1, sizeof(PCPProcessList));
```

and this struct has many pointers to other big structs like:

```
typedef struct ProcessList_ {
   const Settings* settings;

   Vector* processes;         /* all known processes; sort
order can vary and differ from display order */
   Vector* displayList;       /* process tree flattened in
display order (borrowed);
                                 updated in
ProcessList_updateDisplayList when rebuilding panel */
   Hashtable* processTable;   /* fast known process lookup by
PID */

   Hashtable* dynamicMeters;  /* runtime-discovered meters */
   Hashtable* dynamicColumns; /* runtime-discovered Columns
*/
```

And each item in `processes` and `dynamicColumns` represents row in Htop:

```
Process* p = (Process*) Vector_get(this->processes, i);
```

```
DynamicColumn* column = Hashtable_get(dynamicColumns,
fields[i]); // this line invoked when needed to Sort columns
by ...
```

After allocating ProcessList struct, PCP/Htop inter to the main loop whose iterate over all instances of proc.psinfo.pid and start to allocate Process_ and fill each individual entity of it one by one.
Please note this snippet because it's critical, and I think here I will make a lot of changes to implement dynamicTabs:

```
int pid = -1, offset = -1;

/* for every process ... */
while (PCPMetric_iterate(PCP_PROC_PID, &pid, &offset)) {

    bool preExisting;
    Process* proc = ProcessList_getProcess(pl, pid,
&preExisting, PCPProcess_new);
```

`PCPMetric_iterate` function Is a really big loop, doing two important things.

1. It iterates over all instances of PCP_PROC_PID (so luckily It's generic by default, I could use It to iterate any PCP metrics)
2. Return a pointer to the value of the next instance (pid), the pid name is only not a generic name. pid could be just any next instance of any PCP metric.

Now we have the 'key', we will use it to look up for the right process to fill it.
And `ProcessList_getProcess` is just a wrapper for `Hashtable_get`!

Is DynamicColumn generic enough? No.
Why? Because It depends on the struct Process. Please look at the implementation of this function.

```
bool DynamicColumn_writeField(const Process* proc,
RichString* str, unsigned int key);
```

What I am going to do is to find a way to make things more generic and independent from struct Process completely. This is the tricky part and still, I need to find a way to modify the dynamic column hash table.

DynamicTab config files:
Reference: htoprc. This is what I propose:

```
#
# pcp-htop(1) configuration file - see pcp-htop(5)
#

[CGROUP] # screen name
key = CGROUP_CPU_STAT_USER # aka, cgroup.cpu.stat.user
metric, this column will use instead of PCP_PROC_PID, and
will specify the type of this Tab.
columns = MYCGROUP_COLUMN EXECSNOOP_ARGS COLUMN_NAME
sort_key = MYCGROUP
tree_sort_key = PID
tree_view = 0
tree_view_always_by_pid = 0
sort_direction = -1
tree_sort_direction = 1
all_branches_collapsed = 0
```

I think each tab should have metrics from the same InDom (instance domain), because each individual InDom has a different number of instances. And It's logical. even if this InDom has only one metric.
For example cgroup.cpu.stat.user and cgroup.cpu.stat.system could be in the same Tab.
Both have InDom: 3.16
cgroup.subsys.hierarchy and cgroup.cpu.stat.user should not be in the same Tab.
The *'key'* in the config file will specify the InDom for this tab.
The tree view is a nice feature. I need to rethink how I can take advantage of it for all PCP metrics. For now, I might set "tree_view" with neutral values for now, to use later.
Also, If I could make DynamicColumns more generic, I think we will not need columns entities in the config file also. I will explain why next. (but briefly. If the

user could choose and Insert columns from AvailableColumnsPanel.ch, why do We need to specify them in the config file?)

Panel content depends on another panel

Imagine with me. When the user goes to choose a new column from Available columns, and insert it. I don't show the whole available columns!
I want to show just all columns whose InDom is equal to the type of the current tab. Because each tab has a fixed key in the config file as I said before. I think I need to add an id to Screen & columns, this pid == InDom so I could map them together.
Fortunately, this Idea already existed in Htop. there is a relation between the Screens panel & Active columns panel.
I want to make another relation between the Screens panel and the Available Columns panel.
This line is responsible for the relation:

```
ColumnsPanel_fill(this->columns, newFocus->ss,
this->settings->dynamicColumns);
```

Please this demo where `ColumnsPanel_fill` gets invoked:
https://youtu.be/L3SGm6JtaI0

What else?

Maybe there are other problems that will need to be handled by the time that they appear... But these parts are the essential parts that I can see right now.

Porting BPF tools and PCP PMDA

I chose this part to be a part of my journey through the GSoC project.
I am motivated to work on it because I'm interested in performance tools in general, not just Htop. and since I have already ported execsnoop:
https://github.com/performancecopilot/pcp/pull/1565
I think I could port many other BPF tools.
These tools that I think I could port in the GSoC period:
(Simply any tool has 'snoop' in its name because all of them have the same skeleton)
I could probably port a lot more tools than this. will depend on the available time.

Bindsnoop:
https://github.com/iovisor/bcc/blob/master/tools/bindsnoop_example.txt
Biosnoop:
https://github.com/iovisor/bcc/blob/master/tools/biosnoop_example.txt
Drsnoop:
https://github.com/iovisor/bcc/blob/master/tools/drsnoop_example.txt
Exitsnoop:
https://github.com/iovisor/bcc/blob/master/tools/exitsnoop_example.txt
Mountsnoop:
https://github.com/iovisor/bcc/blob/master/tools/mountsnoop_example.txt
Opensnoop:
https://github.com/iovisor/bcc/blob/master/tools/opensnoop_example.txt
Statsnoop:
https://github.com/iovisor/bcc/blob/master/tools/statsnoop_example.txt

Timeline

- June 13 - July 25: Porting Libbpf tools to PCP PMDA
- August 1 - October 31: Implementing Dynamic tabs